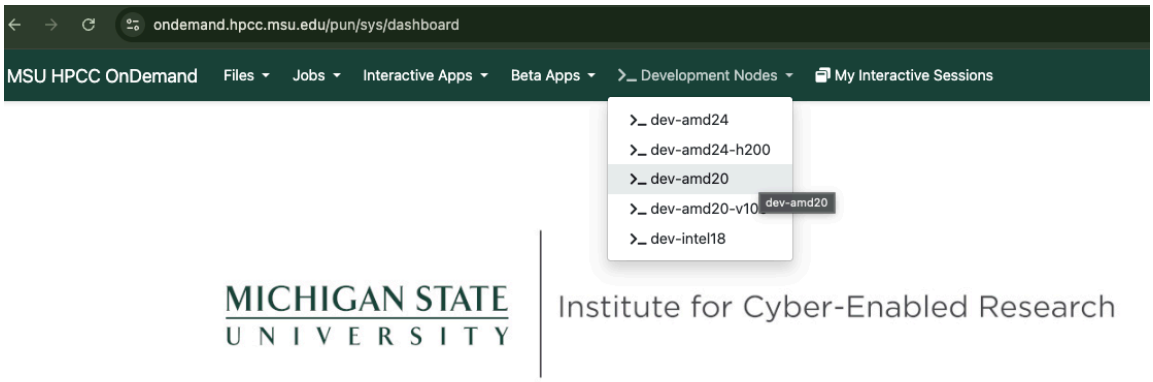
Getting started with Ollama

Starting an interactive job

We'll first login to a dev node terminal through MSU HPCC OnDemand. To do so, you'll simply click on the `>_Development Nodes` dropdown and select a dev node (e.g., dev-amd20).



Next, you can submit for an interactive session on a compute node using the `salloc` command. Here, you'll specify the account (-A) to get placed on compute nodes within our workshop reservation. Each node is on The Data Machine and has four GPUs, each split into seven units. Each of these split units has 10 GB of VRAM. You can submit this job by running the command

```
salloc -A genai_workshop --nodes=1 --time=00:45:00 --mem=16G --cpus-per-task=4 --gpus=a100_slice
```

> Please limit yourself to just a single unit (a100 slice), as we try to accomodate all participants

Once your interactive job starts, you will see it go from pending, through the queue and allocation. Once you see `salloc: Nodes <node-id> are ready for job` you should see a shell has started on that compute node.

```
venegas5@dev-amd20:~$ salloc -A genai_workshop --nodes=1 --time=00:30:00 --mem=16G --cpus-per-task=4 --gpus=a100_slice
salloc: Pending job allocation 3084136
salloc: job 3084136 queued and waiting for resources
salloc: job 3084136 has been allocated resources
salloc: Granted job allocation 3084136
salloc: Waiting for resource configuration
salloc: Nodes nal-005 are ready for job

venegas5@nal-005:~$
```

Loading and running Ollama

Now that you're in an interactive session, you can load the `Ollama` module from our shared module system. You'll run `module load Ollama` to load the module. You can see a short description and some additional information about the Ollama module by running `module show Ollama` in the terminal.

```
venegas5@nal-005:~$ module show Ollama
-------------------------------------------------------------------------------------------------------------------------------------------------------
   /opt/software-current/2023.06/x86_64/generic/modules/all/Ollama/0.15.5.lua:
-------------------------------------------------------------------------------------------------------------------------------------------------------
help([[
Description
===========
Get up and running with large language models. A command-line tool for running large language models locally.


More information
================
 - Homepage: https://ollama.com
]])
whatis("Description: Get up and running with large language models. A command-line tool for running large language models locally.")
whatis("Homepage: https://ollama.com")
whatis("URL: https://ollama.com")
conflict("Ollama")
prepend_path("CMAKE_PREFIX_PATH","/opt/software-current/2023.06/x86_64/generic/software/Ollama/0.15.5")
prepend_path("LIBRARY_PATH","/opt/software-current/2023.06/x86_64/generic/software/Ollama/0.15.5/lib")
prepend_path("PATH","/opt/software-current/2023.06/x86_64/generic/software/Ollama/0.15.5/bin")
setenv("EBROOTOLLAMA","/opt/software-current/2023.06/x86_64/generic/software/Ollama/0.15.5")
setenv("EBVERSIONOLLAMA","0.15.5")
setenv("EBDEVELOLLAMA","/opt/software-current/2023.06/x86_64/generic/software/Ollama/0.15.5/easybuild/Ollama-0.15.5-easybuild-devel")
```

You may begin by running `ollama -h` (or just `ollama`) to see available commands, options and flags.

```
venegas5@nal-005:~$ ollama
Usage:
  ollama [flags]
  ollama [command]

Available Commands:
  serve       Start Ollama
  create      Create a model
  show        Show information for a model
  run         Run a model
  stop        Stop a running model
  pull        Pull a model from a registry
  push        Push a model to a registry
  signin      Sign in to ollama.com
  signout     Sign out from ollama.com
  list        List models
  ps          List running models
  cp          Copy a model
  rm          Remove a model
  launch      Launch an integration with Ollama
  help        Help about any command

Flags:
  -h, --help      help for ollama
  -v, --version   Show version information

Use "ollama [command] --help" for more information about a command.
```

To run ollama commands you'll have to be connected to a running server. If you're not, and you try to run an ollama command (such as ollama list) you will get an error: `Error: could not connect to ollama server, run 'ollama serve' to start it`

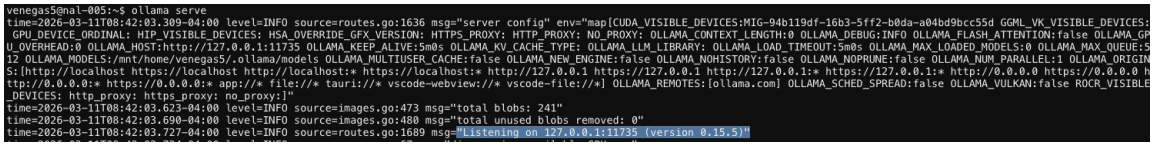
To start and Ollama server, you can run `ollama serve` at which point you may run into a port conflict and get the error: `Error: listen tcp 127.0.0.1:11434: bind: address already in use`

Ollama sets 11434 as the default port number, and with multiple users on the same node you may run into this issue. To avoid this, whenever using Ollama, I recommend changing the port number using the `OLLAMA_HOST` environment variable. You may pick some number (roughly) between 40001 and 40840, and export the set variable. For example, if I want to set the port number to 11735 I would run `export OLLAMA_HOST=127.0.0.1:11735` when starting my session, before starting Ollama. After this, you can try starting Ollama, and once it's started you should see `"Listening on 127.0.0.1:11735 (version 0.15.5)"` in the logs, for whichever port number you selected.

> Note: if you start Ollama on a non-default port number, and you want to use that server in a new shell, you will have to run the OLLAMA_HOST export in the new shell (unless you add it to your shell configuration file).

In short, the full steps to load the Ollama module and start Ollama, using a specified port number, looks like this

```
venegas5@nal-005:~$ module load Ollama
venegas5@nal-005:~$ export OLLAMA_HOST=127.0.0.1:11735
venegas5@nal-005:~$ ollama serve
```



Now that Ollama is started, you can open another dev node terminal and open another shell with the from your currently running interactive session by running. In the new dev node terminal, you'd run `srun --jobid=<your-job-id> --pty bash` and you should see a shell open on the same compute node your Ollama server is running on. First, you'll need to load the Ollama module again, and if you set a different port number when starting the server, you will have to run the export (for `OLLAMA_HOST`) again.

> Note you may have to create the symbolic link to the common model space, if it hasn't yet been created. To do so, you can run the powertool `ollama_link_models` in a dev node terminal.

```
venegas5@nal-005:~$ export OLLAMA_HOST=127.0.0.1:11735
venegas5@nal-005:~$ echo $OLLAMA_HOST
127.0.0.1:11735
venegas5@nal-005:~$ module load Ollama
venegas5@nal-005:~$ ollama list
NAME                                ID              SIZE      MODIFIED
SmolLM3-3B-GGUF:Q4_K_M              67ba6ea15aeb    1.9 GB    23 hours ago
hf.co/ggml-org/SmolLM3-3B-GGUF:Q4_K_M    67ba6ea15aeb    1.9 GB    24 hours ago
gemma3:270m                         e7d36fb2c3b3    291 MB    6 weeks ago
llava:34b                           3d2d24f46674    20 GB     2 months ago
devstral:latest                     9bd74193e939    14 GB     2 months ago
llava:latest                        8dd30f6b0cb1    4.7 GB    2 months ago
codellama:latest                    8fdf8f752f6e    3.8 GB    3 months ago
llama4:scout                        bf31604e25c2    67 GB     3 months ago
gemma3:4b                           a2af6cc3eb7f    3.3 GB    4 months ago
gpt-oss:20b                         17052f91a42e    13 GB     4 months ago
```

You can now pick and run a model. I would recommend a smaller size model (try <8B param models) e.g., `llama3.2:1b` has worked fine w/ these resources. With larger VRAM (e.g., compute nodes w/ h200 GPUs), you can run larger models, but these smaller models serve to demonstrate the basics of using Ollama.

To get more information about the model, such as model architecture, context length, and quantization level, you can use the `model show <model-name>` command.

```
venegas5@nal-005:~$ ollama show llama3.2:1b
  Model
    architecture        llama
    parameters          1.2B
    context length      131072
    embedding length    2048
    quantization        Q8_0

  Capabilities
    completion
    tools

  License
    LLAMA 3.2 COMMUNITY LICENSE AGREEMENT
    Llama 3.2 Version Release Date: September 25, 2024
    ...
```

Configuring Ollama

There are several control options that allow you to configure Ollama and models running with Ollama. A primary set of controls are found in the Ollama environment variables. You can see the available environment variables for `ollama serve` by running `ollama serve --help`

```
venegas5@nal-005:~$ ollama serve --help
Start Ollama

Usage:
  ollama serve [flags]

Aliases:
  serve, start

Flags:
  -h, --help   help for serve

Environment Variables:
      OLLAMA_DEBUG               Show additional debug information (e.g. OLLAMA_DEBUG=1)
      OLLAMA_HOST                IP Address for the ollama server (default 127.0.0.1:11434)
      OLLAMA_CONTEXT_LENGTH      Context length to use unless otherwise specified (default: 4k/32k/256k based on VRAM)
      OLLAMA_KEEP_ALIVE          The duration that models stay loaded in memory (default "5m")
      OLLAMA_MAX_LOADED_MODELS   Maximum number of loaded models per GPU
      OLLAMA_MAX_QUEUE           Maximum number of queued requests
      OLLAMA_MODELS              The path to the models directory
      OLLAMA_NUM_PARALLEL        Maximum number of parallel requests
      OLLAMA_NOPRUNE             Do not prune model blobs on startup
      OLLAMA_ORIGINS             A comma separated list of allowed origins
      OLLAMA_SCHED_SPREAD        Always schedule model across all GPUs
      OLLAMA_FLASH_ATTENTION     Enabled flash attention
      OLLAMA_KV_CACHE_TYPE       Quantization type for the K/V cache (default: f16)
      OLLAMA_LLM_LIBRARY         Set LLM library to bypass autodetection
      OLLAMA_GPU_OVERHEAD        Reserve a portion of VRAM per GPU (bytes)
      OLLAMA_LOAD_TIMEOUT        How long to allow model loads to stall before giving up (default "5m")
```

We will not go into detail with each environment variable, but will use some directly in throughout our demos and exercises (e.g., we already used `OLLAMA_HOST`). You can set these environment variables inline with `ollama serve` or export them before starting Ollama. If you have a set of variables you'd like to keep persistent, you can add them to your `~/.bashrc` file, or you can use a setup shell script whenever you'd like to configure Ollama with pre-defined settings.

In recent Ollama versions, the default context length is tiered by available vram on your machine. When running a model, if the set context length is greater than the context length the model was trained on, it will be set to the training context length. Assuming you're running on one of the reserved nodes [nal-005, nal-006, nal-007], with an a100 slice (10gb vram), you will likely run into issues trying to load a model like `llama3.2:3b` without configuring the context length. In this example, you may see an out of memory error, completely unable to run the model.

```
venegas5@nal-005:/mnt/research/HPCC_AI_Workshop$ ollama run llama3.2:3b
Error: 500 Internal Server Error: llama runner process has terminated: cudaMalloc failed: out of memory
alloc_tensor_range: failed to allocate CUDA0 buffer of size 15032385536
```

There are other several controls you may use to get this model running within the vram limits, and a major one is with the `OLLAMA_CONTEXT_LENGTH` environment variable. If you start the server with the context length set lower (e.g., 4096), you will likely be able to run the model without the memory error. To do this, you could run `OLLAMA_CONTEXT_LENGTH=4096 ollama serve` and then run the model. In the server logs you should see `n_ctx` is now set to 4096, and running `ollama ps` you should see the updated context.

```
venegas5@nal-005:/mnt/research/HPCC_AI_Workshop$ ollama ps
NAME           ID              SIZE      PROCESSOR    CONTEXT    UNTIL
llama3.2:3b    a80c4f17acd5    2.8 GB    100% GPU     4096       3 minutes from now
```

If you need a larger context length for your tasks, another way you can work within vram limits is with the `OLLAMA_KV_CACHE_TYPE` environment variable. This sets quantization for KV (Key-Value) caching. The quantization essentially compresses stored attention keys and values (major component of Transformers) from high precision (Fp16) to lower precision (8-bit, 4-bit). The trade-off is of course, lower precision in model responses. This is often used with Flash Attention enabled (`OLLAMA_FLASH_ATTENTION=1`).

> Note KV cache quantization is not the inference quantization of a given model (weights for inference).

For example, you can start Ollama with flash attention enabled and using 4-bit KV cache quantization by running `OLLAMA_FLASH_ATTENTION=1 OLLAMA_KV_CACHE_TYPE=q4_0 ollama serve`

You should now be able to run `llama3.2:3b` with the default (max) context length.

```
venegas5@nal-005:/mnt/research/HPCC_AI_Workshop$ ollama ps
NAME           ID              SIZE     PROCESSOR    CONTEXT    UNTIL
llama3.2:3b    a80c4f17acd5    12 GB    100% GPU     131072     4 minutes from now
```